# System Architecture

A complete map of the PhysioGuidance production system: services, data stores, integrations, security layers, and how the pieces connect on Google Cloud Platform in the me-central2 (Dammam) region.

PREPARED FOR

**PhysioGuidance Client Team**

DATE

**2026-08-17**

DEPLOYED COMMIT

`d396b040`

**PHYSIOGUIDANCE**

Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

# Table of contents


1. Executive summary
2. Bird's-eye topology
3. Component inventory
4. Data flow — how a request is served
5. Data flow — how background work is scheduled and run
6. Data flow — how a payment settles
7. Data stores
8. Storage buckets
9. Networking and security boundaries
10. Regionality and data residency
11. Deployment topology
12. Component status matrix


---

# 1. Executive summary

PhysioGuidance is a multi-tenant learning platform delivered as five Cloud Run services in Google Cloud's Saudi region (me-central2):

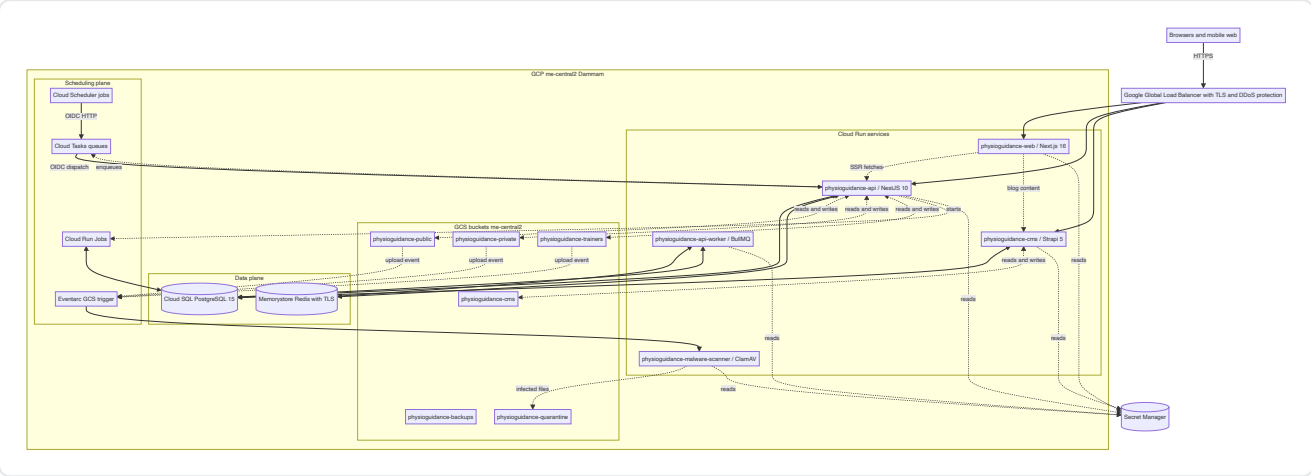
- **Frontend web:** Next.js 16 App Router on Node 20, with `[locale]` prefix for EN and AR.
- **Backend API:** NestJS 10 on Node 20, REST with JWT auth and Drizzle ORM against PostgreSQL 15.
- **Async worker:** the same NestJS image, run with BullMQ consumers active and in-container crons off, on a separate Cloud Run service.
- **CMS:** Strapi 5 for blog and marketing content, backed by the same Cloud SQL instance.
- **Malware scanner:** ClamAV in Node 20, deployed to Cloud Run and driven by Eventarc from GCS upload events.

Persistent state lives in Cloud SQL (PostgreSQL 15) and six GCS buckets. Ephemeral state (BullMQ queues, rate-limit counters, distributed locks) lives in Memorystore Redis reached over a private VPC connector with TLS. Time-shifted work runs in three flavours: Cloud Tasks for event-driven single-shot deferrals, Cloud Scheduler for calendar-driven jobs, and Cloud Run Jobs for heavy per-execution containers (video transcode, caption generation, WCAG checks).

Every production resource runs in **me-central2** — mandated by Saudi PDPL data-residency, NELC accreditation, and SDAIA registration.

---

# 2. Bird's-eye topology




**How to read this:** solid lines are the request/response path. Dotted lines are asynchronous, event-driven, or config lookups. Everything inside the shaded box runs in the me-central2 region.

# 3. Component inventory

| Service | Type | Runtime | Purpose | Public URL |
|---|---|---|---|---|
| `physioguidance-web` | Cloud Run service | Next.js 16.1 on Node 20 | Student, instructor, admin, finance, and support UIs | https://physioguidance.org |
| `physioguidance-api` | Cloud Run service | NestJS 10 on Node 20 | REST API for every client, JWT auth, all business logic | https://api.physioguidance.org |
| `physioguidance-api-worker` | Cloud Run service | Same NestJS image, workers-on / crons-off | BullMQ consumers for webhooks, xAPI, media pipeline chaining | internal only |
| `physioguidance-cms` | Cloud Run service | Strapi 5.35 on Node 20 | Blog CMS and editorial workflow | https://cms.physioguidance.org |
| `physioguidance-malware-scanner` | Cloud Run service | ClamAV plus Node 20 | Malware scanning on GCS uploads, invoked by Eventarc | internal only |
| `physioguidance-db-me` | Cloud SQL | PostgreSQL 15 | All persistent business data | private IP plus Cloud SQL Auth Proxy |
| Memorystore Redis | Memorystore | Redis | BullMQ queues, distributed locks, rate-limit counters | private VPC only |
| GCS buckets, six of them | Cloud Storage | Regional, single-region | See section 8 for per-bucket detail | signed URLs or streaming via the API |

Cloud Run sizing baseline, from `infra/config.sh`:

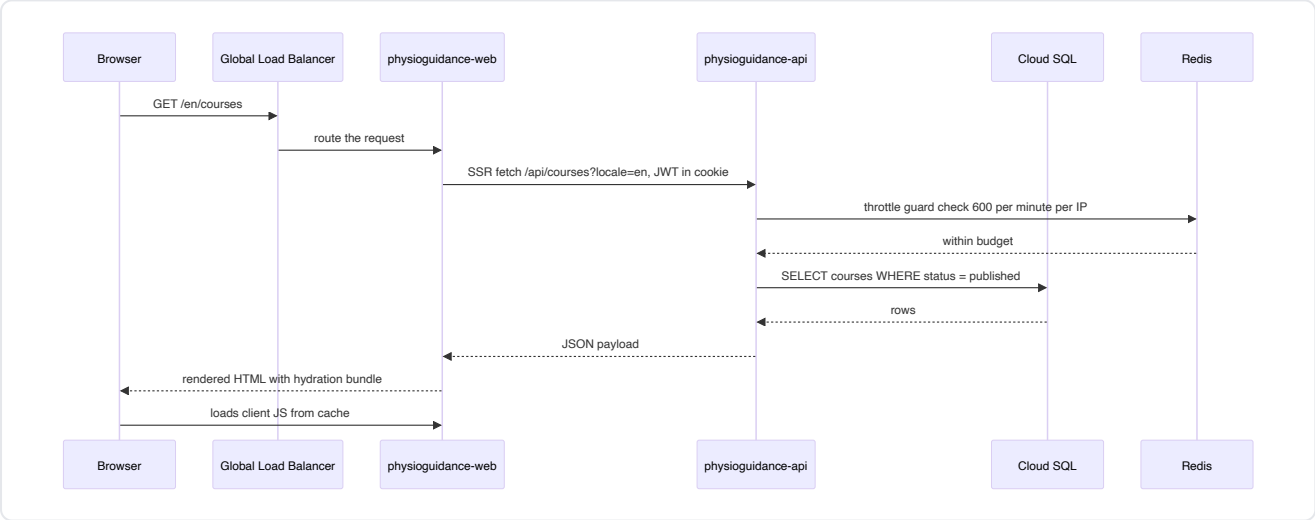
| Service | CPU | Memory | min / max instances |
|---|---|---|---|
| `physioguidance-web` | 1 vCPU | 512 MiB | 0 / 3 |
| `physioguidance-api` | 1 vCPU | 1 GiB | 0 / 10 |
| `physioguidance-api-worker` | 1 vCPU | 1.5 GiB | 1 / 1 |
| `physioguidance-cms` | 1 vCPU | 512 MiB | 0 / 2 |
| `physioguidance-malware-scanner` | 2 vCPU | 4 GiB | 0 / 5 |

The worker runs `min=max=1` with CPU always allocated so BullMQ can poll Redis continuously; the API runs with CPU-throttling on (request-scoped billing).

---

# 4. Data flow — how a request is served

Example: a student loads the course catalog.




**Key details verified in code:**

- Every non-`@Public()` API route enforces JWT auth via `JwtAuthGuard`, registered globally in `app.module.ts`.
- Every route also passes through `ThrottlerGuard` at 600 requests per minute per IP (`ttl: 60000, limit: 600`).
- The web service runs server-side rendering, so first paint arrives as fully-rendered HTML with the hydration bundle attached.
- Locale (`en` or `ar`) is always a path prefix under the `[locale]` App Router segment, never a query parameter or header.

---

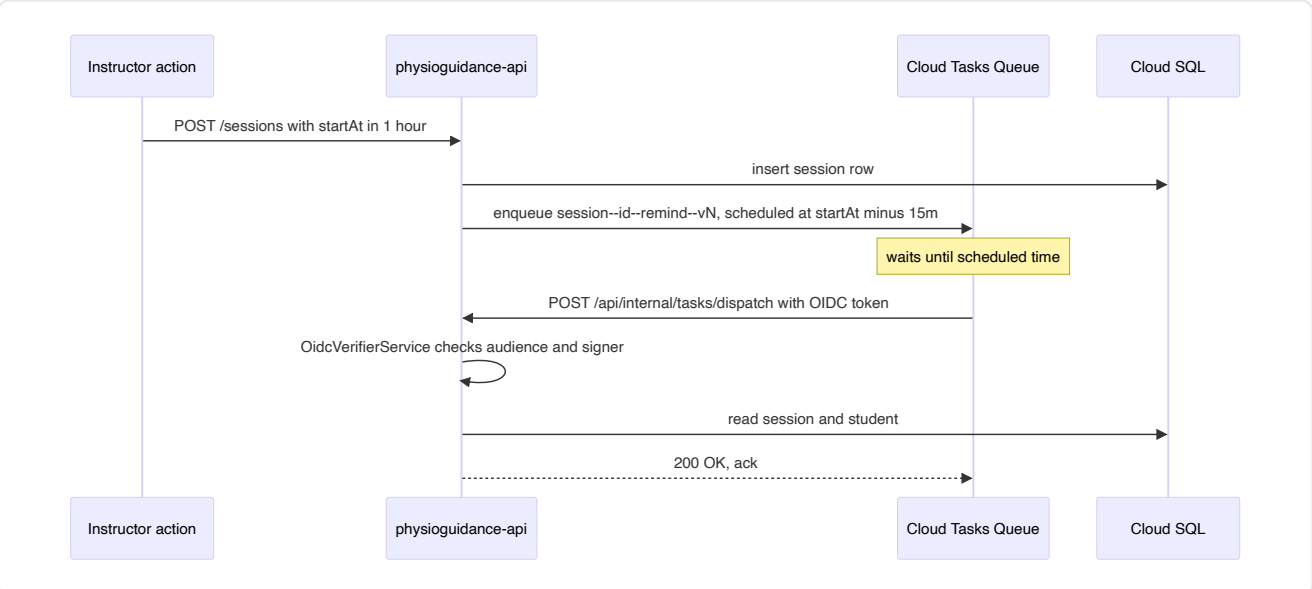
# 5. Data flow — how background work is scheduled and run

Three flavours are in production:

1. **Cloud Tasks** for event-driven deferrals: session reminders, ticket auto-close, payment cleanup, and any per-row lifecycle event. The API enqueues a task with a deterministic name and OIDC-signed dispatch. Handlers live in the API service under `backends/src/modules/*/*.tasks.ts` ( `payments` , `sessions` , `support` , `points` , `backups` , `scheduled-maintenance` ).
2. **Cloud Scheduler** for calendar-driven jobs: daily payment reconciliation, stale payment sweep, and BNPL reconciliation, all provisioned by `infra/scripts/setup-reconciliation-cron.sh` .
3. **Cloud Run Jobs** for heavy per-execution containers: `physio-job-video-transcode` , `physio-job-captions-generate` , and `physio-job-wcag-check` . Provisioned by `infra/scripts/setup-{video-transcode,captions-generate,wcag-check}-job.sh` and started from the API through `CloudRunJobsService.runJob(...)` .

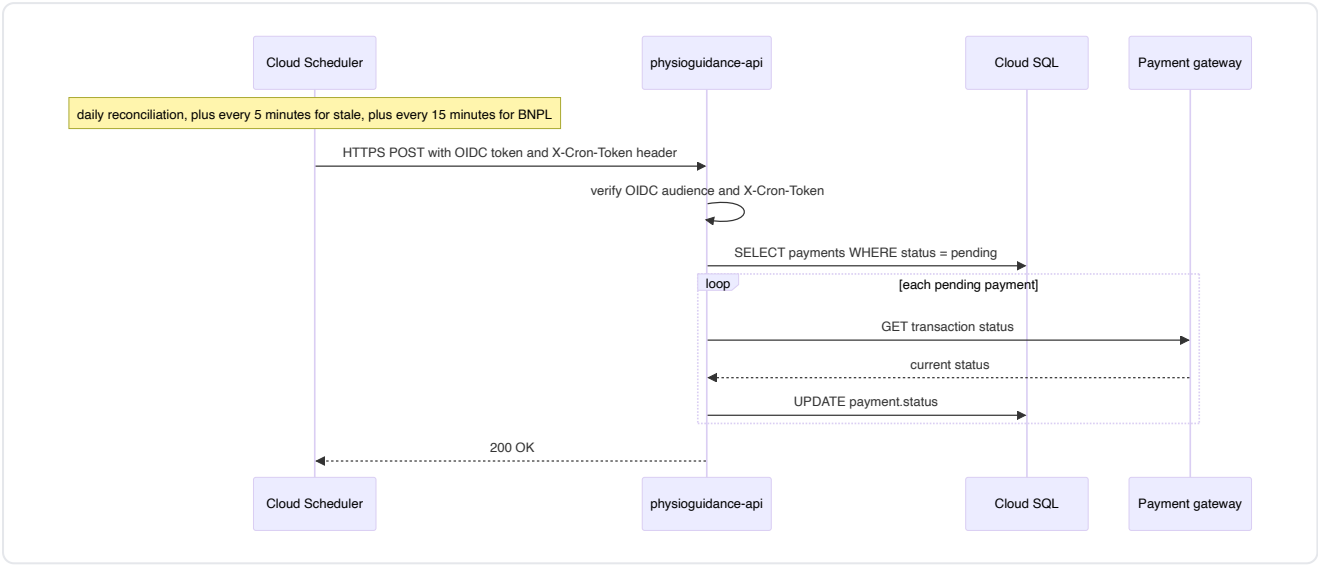
Time-triggered work that stays lightweight and in-process still uses `@nestjs/schedule` decorators inside the API service; the worker is deployed with `SKIP_CRONS=true` so no decorator ever double-fires.

## 5.1 Event-driven deferral: a session reminder




Cloud Tasks deduplicates by task name globally per queue, so deterministic names like `session--{id}--remind--v{updatedAt}` make the enqueue-on-write pattern race-safe: any concurrent update naturally invalidates the prior task name and re-schedules cleanly.

### 5.2 Calendar-driven job: payment reconciliation



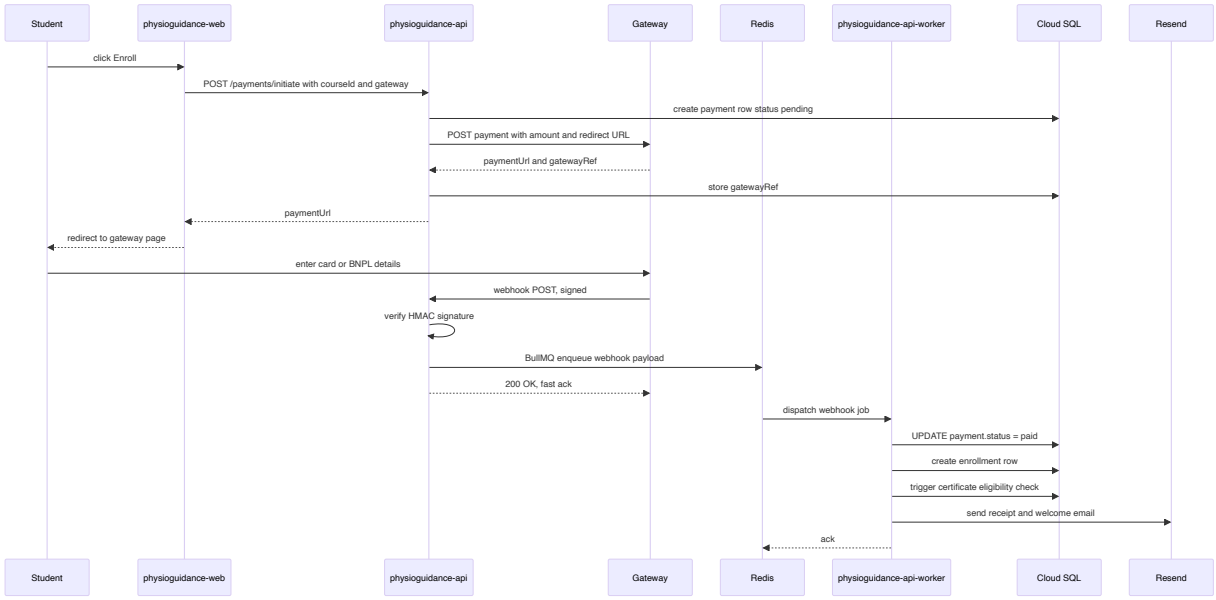

### 5.3 Heavy processing: video upload pipeline

An instructor upload starts `physio-job-video-transcode` (ffmpeg on 2 vCPU / 2 GiB, up to 90 minutes) as a Cloud Run Job. On success it chains `physio-job-captions-generate` and then `physio-job-wcag-check` through BullMQ handoff on Memorystore. Idle cost stays near zero; execution cost is paid only during actual runs.

---

## 6. Data flow — how a payment settles

The most business-critical flow. It involves the frontend, backend, an external gateway (Moyasar, Tabby, or Tamara), Memorystore for the webhook queue, Cloud SQL, and the async worker.




**Reliability model:**

- **Webhook lost:** Cloud Scheduler reconciliation (5 minutes for stale, daily for the full sweep, 15 minutes for BNPL) re-fetches gateway status and closes the loop.
- **Redis restart:** in-flight webhook jobs are lost by design, but reconciliation catches up. See redis-persistence-risk.md.
- **HMAC verification fails:** request rejected with 401 and logged to Sentry.

---

# 7. Data stores

## 7.1 Cloud SQL — `physioguidance-db-me`

| Property | Value |
|---|---|
| Engine | PostgreSQL 15 |
| Tier | `db-g1-small` (1 vCPU, 1.7 GiB RAM) |
| Region | me-central2 |
| Access | Private IP plus Cloud SQL Auth Proxy over the `physio-vpc-connector-me` VPC connector |
| Schema management | Drizzle ORM, migrations under `backends/src/database/migrations/` (126 applied) |
| Backups | Managed Cloud SQL automated backups, retention configured on the instance |

All application data lives in a single `physioguidance` database. Domain schemas (auth, courses, payments, certificates, materials, and so on) are all Drizzle-managed. The Strapi CMS uses the same Cloud SQL instance for its own tables.

### 7.2 Memorystore Redis

| Property | Value |
|---|---|
| Access | Private VPC via `physio-vpc-connector-me`, port 6378, TLS enabled |
| Data held | BullMQ queues, distributed locks, rate-limit counters |
| Persistence posture | Ephemeral by design; every durable fact lives in Cloud SQL |

Redis holds no user data. All processes reconnect on cold start; Cloud Scheduler reconciliation covers any in-flight work that vanishes during a Memorystore restart. Detail in redis-persistence-risk.md.

---

## 8. Storage buckets

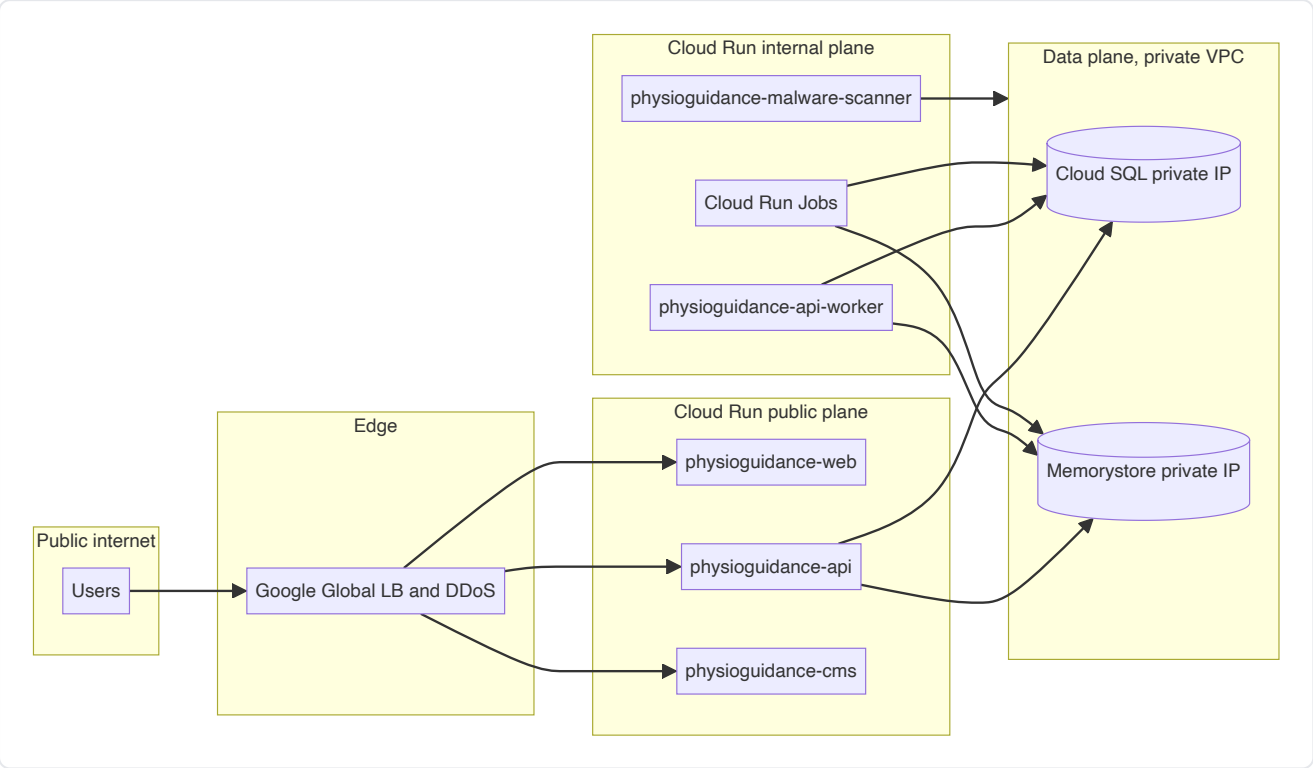
Six regional buckets, all in me-central2:

| Bucket | Purpose | Access pattern | Malware scan |
|---|---|---|---|
| `physioguidance-public` | Publicly viewable assets: transcoded materials, avatars, blog images | Public via signed URL or streaming | Scanned by Eventarc trigger |
| `physioguidance-private` | Payment receipts, bank-transfer proofs, PDPL exports | Signed URL only, download gate on scan verdict | Scanned by Eventarc trigger |
| `physioguidance-trainers` | Trainer CVs, credentials, certificates | Signed URL only, download gate on scan verdict | Scanned by Eventarc trigger |
| `physioguidance-cms` | Strapi media, uploaded from the CMS through `@strapi-community/strapi-provider-upload-google-cloud-storage` | Public read via Strapi | Not on the scanner allowlist by design (editorial content only) |
| `physioguidance-backups` | Cloud SQL SQL exports for the admin Backups tab | Admin and operators only, 30-day lifecycle delete | Not scanned (system-generated) |
| `physioguidance-quarantine` | Files quarantined by the scanner | Ops only | Terminal state |

Bank-transfer receipts inside `physioguidance-private` carry a longer lifecycle rule set by `setup-bank-receipt-retention.sh` (Saudi accounting law).

---

# 9. Networking and security boundaries




**Boundaries:**

- Three services face the internet ( `web` , `api` , `cms` ). The worker and scanner are deployed with `--no-allow-unauthenticated` and take no public traffic.
- Cloud SQL and Memorystore have private IPs only; every process reaches them through the `physio-vpc-connector-me` VPC connector with `vpc-egress=all-traffic` .
- Every internet-facing request passes through Google's edge: TLS termination, built-in DDoS mitigation, IPv4 and IPv6.
- App-layer defenses: NestJS `ThrottlerGuard` at 600 req/min/IP, argon2 password hashing, Drizzle parameterized queries, reCAPTCHA on login and register, HMAC verification on every gateway webhook, PII field-level encryption ( `PII_ENCRYPTION_KEY` from Secret Manager) on `national_id` , `phone` , and `scfhs_number` . See security-architecture.md.
- Every Cloud Run service runs as its own runtime service account ( `api-runtime` , `worker-runtime` , `malware-runtime` , and so on), granted only the roles that service needs.
- Cloud Tasks dispatch is protected two ways: OIDC audience match verified by `OidcVerifierService` , and an `X-Cron-Token` header from Secret Manager on cron endpoints as a belt-and-braces layer.

# 10. Regionality and data residency

Every production resource is regional in **me-central2** (Dammam, Saudi Arabia):

| Class | Region |
|---|---|
| Cloud Run services (5) | me-central2 |
| Cloud Run Jobs (3) | me-central2 |
| Cloud SQL | me-central2 |
| Memorystore Redis | me-central2 |
| GCS buckets (6) | me-central2 |
| Cloud Tasks, Cloud Scheduler, Eventarc | me-central2 |
| VPC connector `physio-vpc-connector-me` | me-central2 |
| Artifact Registry `physioguidance` | me-central2 |
| Secret Manager | Auto-replicated, Google-managed |

**Why:** Saudi PDPL Article 29, SDAIA registration, and NELC accreditation all require customer PII to remain in-country. The platform runs in me-central2 by design. The regional-outage playbook is in regional-failure-playbook.md.

# 11. Deployment topology

| Path | What it deploys | Trigger |
|---|---|---|
| `.github/workflows/deploy-backend.yml` | Builds and deploys `physioguidance-api` | Push to `main` touching `backends/**`, `infra/backend/**`, or `infra/config.sh` |
| `.github/workflows/deploy-frontend.yml` | Builds and deploys `physioguidance-web` | Push to `main` touching `frontend/**` |
| `.github/workflows/deploy-worker.yml` | Deploys `physioguidance-api-worker` on the current backend image | Chained after a successful backend deploy |
| `.github/workflows/deploy-strapi.yml` | Builds and deploys `physioguidance-cms` | Push to `main` touching `strapi/**` |
| `infra/malware-scanner/deploy.sh` | Builds ClamAV image, deploys the scanner, wires or refreshes Eventarc triggers | Manual (rare — stable component; re-run to refresh signatures) |
| `infra/scripts/setup-{video-transcode,captions-generate,wcag-check}-job.sh` | Provisions the three Cloud Run Jobs on the current backend image | Manual on schema or resource change |
| `infra/scripts/setup-cloud-tasks.sh` | Provisions the eight Cloud Tasks queues | Manual, idempotent |
| `infra/scripts/setup-reconciliation-cron.sh` | Provisions the three Cloud Scheduler jobs | Manual, idempotent |

**Deploy identity:** GitHub Actions authenticates to GCP via Workload Identity Federation (`WIF_PROVIDER` and `WIF_SERVICE_ACCOUNT` secrets), so no long-lived service-account keys ever live in GitHub. Each workflow uses a per-service runtime SA when it calls `gcloud run deploy`.

**Rollback:** Cloud Run console → service → Revisions → previous revision → Manage traffic → 100%. Takes about 10 seconds. The backend deploy script pins `--to-latest` after every push, so a rollback to a named revision remains authoritative until the next deploy.

Full detail in cicd-deployment.md and gcp-deployment.md.

# 12. Component status matrix

Using Amna's four-category vocabulary.

| Component | Status | Notes |
|---|---|---|
| Frontend web (Next.js 16) | Implemented | Live at `https://physioguidance.org` |
| Backend API (NestJS 10) | Implemented | Live at `https://api.physioguidance.org`, ThrottlerGuard active at 600 req/min/IP |
| Async worker (BullMQ) | Implemented | Live, min=max=1, no-cpu-throttling, workers-on and crons-off |
| Strapi CMS (5.35) | Implemented | Live at `https://cms.physioguidance.org`, media on `physioguidance-cms` bucket |
| Malware scanner (ClamAV) | Implemented | 4 GiB / 2 vCPU, watching `physioguidance-public`, `physioguidance-private`, `physioguidance-trainers`; quarantines to `physioguidance-quarantine` |
| Cloud SQL PostgreSQL 15 | Implemented | `physioguidance-db-me`, `db-g1-small`, private IP over VPC connector |
| Memorystore Redis | Implemented | Private VPC, TLS on port 6378 |
| Cloud Tasks | Implemented | Eight queues provisioned by `setup-cloud-tasks.sh`; task handlers under `backends/src/modules/*/*.tasks.ts` |
| Cloud Scheduler | Implemented | Three reconciliation jobs (daily, stale, BNPL) provisioned by `setup-reconciliation-cron.sh` |
| Cloud Run Jobs | Implemented | `physio-job-video-transcode`, `physio-job-captions-generate`, `physio-job-wcag-check` |
| Per-service runtime SAs | Implemented | `api-runtime`, `worker-runtime`, `malware-runtime`, `tasks-dispatcher`, and per-Job SAs |
| Malware download gate | Implemented | Private and trainers buckets gate downloads on scan verdict |
| Field-level PII encryption | Implemented | `PII_ENCRYPTION_KEY` from Secret Manager on `national_id`, `phone`, `scfhs_number` |
| Cloud Armor / WAF | Not Applicable | Amna doc scopes this documentation-only; Google edge protection plus app-layer defenses in place |

## 13. Where to look next

- **How each service is deployed and configured** → gcp-deployment.md
- **How code gets from git to production** → cicd-deployment.md
- **Security posture, threat model, controls** → security-architecture.md
- **What happens when things break** → regional-failure-playbook.md
- **PDPL and NELC compliance** → pdpl-compliance.md and nelc-compliance.md
- **Day-to-day operations** → platform-owner-guide.md